

App Support Socket Chat Integration

Documentation

This document describes the real-time WebSocket events for the **App Support Chat System**. It integrates both the regular User (mobile/client app) and the Administrator (Support Web Dashboard).

1. Authentication

All socket connections are authenticated via the `socketAuthMiddleware` . The developer must supply the JWT token during connection initialization:

- **Authorization Header:**

```
const socket = io("wss://api.ridewithpals.com/", {
  extraHeaders: {
    Authorization: "Bearer <JWT_TOKEN>"
  }
});
```

- **Query Parameter:**

```
const socket = io("wss://api.ridewithpals.com/", {
  query: {
    token: "Bearer <JWT_TOKEN>"
  }
});
```

2. Event Directory Reference

Event Name	Role	Type	Description
<code>support:thread:getOrCreate</code>	User	Emitted	Retrieves or initializes the user's support thread
<code>support:thread:join</code>	User & Admin	Emitted	Joins the real-time socket room of a thread

Event Name	Role	Type	Description
<code>support:thread:leave</code>	User & Admin	Emitted	Leaves the thread socket room
<code>support:message:send</code>	User & Admin	Emitted	Sends a message to the active support thread
<code>support:messages:list</code>	User & Admin	Emitted	Retrieves paginated message history
<code>support:threads:list</code>	Admin	Emitted	Lists all open/assigned support threads with unread counts
<code>support:thread:read</code>	User & Admin	Emitted	Marks all unread incoming messages as read
<code>support:message:new</code>	User & Admin	Listen	Fired when a new message arrives in the room
<code>support:thread:read</code>	User & Admin	Listen	Fired when a participant marks the thread as read

3. Detailed Event Reference

`support:thread:getOrCreate`

- **Initiated By:** User (Client App)
- **Payload:** *None (Securely determined via connection token)*
- **Ack Response:**

```
{
  "ok": true,
  "data": {
    "id": 12,
    "userId": 45,
    "adminId": null,
    "status": "open",
    "lastMessageAt": "2026-07-14T12:00:00.000Z",
    "createdAt": "2026-07-14T12:00:00.000Z",
    "updatedAt": "2026-07-14T12:00:00.000Z"
  }
}
```

(Note: The user automatically joins the support room `support:thread:12` on triggering this)

support:thread:join

- **Initiated By:** User or Admin
- **Description:** Subscribes the socket session to the support room. If an **Admin** joins an unassigned thread (status `"open"`), it automatically claims the ticket (assigning `adminId` and setting status to `"assigned"`).
- **Payload:**

```
{
  "threadId": 12
}
```

- **Ack Response:**

```
{
  "ok": true
}
```

support:thread:leave

- **Initiated By:** User or Admin
- **Payload:**

```
{
  "threadId": 12
}
```

- **Ack Response:**

```
{
  "ok": true
}
```

support:message:send

- **Initiated By:** User or Admin
- **Description:** Sends a text message to the thread. If the admin sends a message and the user is offline/not in the socket room, it triggers an offline Push Notification automatically.
- **Payload:**

```
{
  "threadId": 12,
  "message": "Can you help me upgrade my subscription plan?"
}
```

- **Ack Response:**

```
{
  "ok": true,
  "data": {
    "id": 142,
    "threadId": 12,
    "senderId": 45,
    "senderType": "user",
    "message": "Can you help me upgrade my subscription plan?",
    "isRead": false,
    "createdAt": "2026-07-14T12:01:00.000Z",
    "updatedAt": "2026-07-14T12:01:00.000Z"
  }
}
```

support:messages:list

- **Initiated By: User or Admin**
- **Payload:**

```
{
  "threadId": 12,
  "limit": 20,
  "offset": 0
}
```

- **Ack Response:**

```
{
  "ok": true,
  "data": {
    "count": 1,
    "rows": [
      {
        "id": 142,
```

```
        "threadId": 12,
        "senderId": 45,
        "senderType": "user",
        "message": "Can you help me upgrade my subscription plan?",
        "isRead": false,
        "createdAt": "2026-07-14T12:01:00.000Z"
      }
    ]
  }
}
```

support:threads:list

- **Initiated By: Admin**
- **Description:** Returns a list of all active support tickets, including requesting user details, the latest message info, and `unreadCount` to display unread badges (e.g. Ali Khan: 2 unread).
- **Payload:**

```
{
  "limit": 20,
  "offset": 0
}
```

- **Ack Response:**

```
{
  "ok": true,
  "data": {
    "count": 1,
    "rows": [
      {
        "id": 12,
        "userId": 45,
        "adminId": 1,
        "status": "assigned",
        "lastMessageAt": "2026-07-14T12:01:00.000Z",
        "user": {
          "id": 45,
          "fullName": "Muhammad Saif",
          "profileImage": "https://...",
          "email": "saif@example.com"
        }
      },
    ],
  }
}
```

```
    "lastMessage": {
      "id": 142,
      "message": "Can you help me upgrade my subscription plan?",
      "createdAt": "2026-07-14T12:01:00.000Z"
    },
    "unreadCount": 1
  }
]
}
}
```

support:thread:read

- **Initiated By:** User or Admin
- **Description:** Marks all unread incoming messages in the thread as read. Emits a `support:thread:read` event in the room.
- **Payload:**

```
{
  "threadId": 12
}
```

- **Ack Response:**

```
{
  "ok": true
}
```

4. Incoming Broadcast Events (Listening)

support:message:new

- **Recipient:** User & Admin
- **Triggered When:** A new message is successfully sent to the room.
- **Incoming Payload:**

```
{
  "threadId": 12,
  "message": {
```

```
    "id": 142,  
    "threadId": 12,  
    "senderId": 45,  
    "senderType": "user",  
    "message": "Can you help me upgrade my subscription plan?",  
    "isRead": false,  
    "createdAt": "2026-07-14T12:01:00.000Z"  
  }  
}
```

support:thread:read (Broadcast)

- **Recipient:** User & Admin
- **Triggered When:** A participant marks unread messages in the thread as read.
- **Incoming Payload:**

```
{  
  "threadId": 12,  
  "readerId": 45,  
  "role": "user"  
}
```

5. Step-by-Step Example Chat Flow

Here is a full step-by-step example of how a User (App client) starts a conversation, sends a message, and how the Admin (Dashboard agent) interacts with it in real-time.

Step 1: User Opens Support Chat

1. **Action:** The User connects to the socket and requests their support ticket room:
 - **Event Name:** `support:thread:getOrCreate`
 - **Payload:** `{}`
2. **Server Action:** Checks DB for User ID `5` (e.g. `Muhammad Saif`), retrieves/creates their thread (`id: 12`), joins user's socket to room `support:thread:12`, and returns thread details.
3. **Response Received by User:**

```
{  
  "ok": true,
```

```
"data": {
  "id": 12,
  "userId": 5,
  "status": "open"
}
```

Step 2: User Sends a Message

1. **Action:** The User sends their chat message:

- **Event Name:** `support:message:send`
- **Payload:**

```
{
  "threadId": 12,
  "message": "How do I upgrade my subscription?"
}
```

2. **Server Action:**

- Saves message to DB (`senderType: "user"`).
- Updates `lastMessageAt` on `SupportThread (id: 12)` .
- Broadcasts the message to the room `support:thread:12` :
 - **Listen Event:** `support:message:new`
 - **Incoming Payload:**

```
{
  "threadId": 12,
  "message": {
    "id": 101,
    "senderId": 5,
    "senderType": "user",
    "message": "How do I upgrade my subscription?"
  }
}
```

Step 3: Admin Dashboard Receives the Message

1. **Action (List):** The Admin's active dashboard panel regularly requests the tickets list:

- **Event Name:** `support:threads:list`
- **Payload:** { `"limit": 10, "offset": 0` }

2. Response Received by Admin:

```
{
  "ok": true,
  "data": {
    "count": 1,
    "rows": [
      {
        "id": 12,
        "userId": 5,
        "user": { "fullName": "Muhammad Saif" },
        "lastMessage": { "message": "How do I upgrade my subscription" },
        "unreadCount": 1
      }
    ]
  }
}
```

The admin dashboard sidebar updates the unread badge count for Muhammad Saif to **1**.

Step 4: Admin Selects the Ticket and Joins

1. **Action:** The admin clicks the conversation in their sidebar to open it:
 - **Event Name:** `support:thread:join`
 - **Payload:** `{ "threadId": 12 }`
2. **Server Action:** Joins the admin's socket to room `support:thread:12`, updates the thread status to `"assigned"`, and assigns the admin's ID.
3. **Action (Mark Read):** The admin reads the chat window and clears the unread count:
 - **Event Name:** `support:thread:read`
 - **Payload:** `{ "threadId": 12 }`
4. **Server Action:** Updates all unread messages from `senderType: "user"` in thread `12` to `isRead: true` and broadcasts read confirmation:
 - **Listen Event:** `support:thread:read`
 - **Incoming Payload:** `{ "threadId": 12, "readerId": 1, "role": "admin" }`

Step 5: Admin Replies

1. **Action:** The Admin types their reply and sends it:
 - **Event Name:** `support:message:send`
 - **Payload:**

```
{
  "threadId": 12,
  "message": "Let me check on that right away."
}
```

2. **Server Action:**

- Saves message to DB(`senderType: "admin"`).
- Broadcasts `support:message:new` to room `support:thread:12` containing the new message.

3. **Result:** The User (active in the support chat window) immediately receives the event and renders the reply!